CERTIK

Cetus - Stable Farming

Security Assessment

CertiK Assessed on Oct 20th, 2026





CertiK Assessed on Oct 20th, 2026

Cetus - Stable Farming

The security assessment was prepared by CertiK.

Executive Summary

TYPES	ECOSYSTEM	METHODS
Staking	Sui (SUI)	Manual Review, Static Analysis

LANGUAGE	TIMELINE
Move	Preliminary comments published on 11/12/2025
	Final report published on 10/20/2026

Vulnerability Summary



16	10	0	6	0
Total Findings	Resolved	Partially Resolved	Acknowledged	Declined

1	Centralization	1 Acknowledged	Centralization findings highlight privileged roles & functions and their capabilities, or instances where the project takes custody of users' assets.
0	Critical		Critical risks are those that impact the safe functioning of a platform and must be addressed before launch. Users should not invest in any project with outstanding critical risks.
0	High		High risks may include logical errors that, under specific circumstances, could result in fund losses or loss of project control.
2	Medium	2 Resolved	Medium risks may not pose a direct risk to users' funds, but they can affect the overall functioning of a platform.
4	Low	1 Resolved, 3 Acknowledged	Low risks can be any of the above, but on a smaller scale. They generally do not compromise the overall integrity of the project, but they may be less efficient than other solutions.
9	Informational	7 Resolved, 2 Acknowledged	Informational errors are often recommendations to improve the style of the code or certain operations to fall within industry best practices. They usually do not affect the overall functioning of the code.

TABLE OF CONTENTS | CETUS - STABLE FARMING

Audit Summary

Executive Summary

Vulnerability Summary

Codebase

Audit Scope

Approach & Methods

Review Notes

Overview

External Dependencies

Privileged Roles

Upgradeability

Findings

CSF-08 : Centralization Related Risks And Upgradability

CSF-09 : Stale Pool Last_reward_time When Emission=0 Causes Retroactive Rewards

CSF-11 : `linked_table.next()` On A Removed Key Can Aborts

CSF-12 : Inconsistency In Updates Made By `update_effective_tick_range`

CSF-13 : The Risk Of Insufficient Reward Funds

CSF-14 : Potential Stalled Traversal

CSF-19 : Fixed-Coin Liquidity Addition Accepts Unbounded Liquidity And Price Outcomes

CSF-03 : Discussion On "Set_package_version()"

CSF-04 : Potential Loss Of The Payout

CSF-05 : Discussion On Pausing Of `clmm_pool`

CSF-06 : Potential Inconsistent Price Source During Post-Withdraw Validation And Share Accounting

CSF-07 : Concerns About Share Calculation Rounds Up

CSF-10 : Several Issues With Migrating Invalid Pool Positions

CSF-15 : Wrong Error Code

CSF-16 : Wrong `UpdateEffectiveTickRangeEvent` End

CSF-17 : Potential CLMM Pool Mismatch In Migration

Optimizations

CSF-01 : Lack `clmm_pool` id check

CSF-02 : High Gas Consumption Issues in Pool and Rewarder

Appendix

Disclaimer

CODEBASE | CETUS - STABLE FARMING

Repository

https://github.com/CetusProtocol/cetus-clmm-sui/tree/main/stable_farming

Commit

0e288bfd710883e9dc48fff439cb6c62b95e57ce

ac44249508a51947cbfaf6d6cf048496328d3d1b

fa08737a605c78c2aa77fe31fb864878c732e469

14fadb735e3ec744721fe65e05f72554045edeca

573d537888d6f6b6f5aa5ce77d8c61c875cc396f

AUDIT SCOPE | CETUS - STABLE FARMING

CetusProtocol/cetus-clmm-sui
sources/pool.move
sources/rewarder.move
sources/pool.move
sources/config.move
sources/acl.move
sources/router.move
sources/acl.move
sources/config.move
sources/rewarder.move
sources/router.move

APPROACH & METHODS | CETUS - STABLE FARMING

This audit was conducted for Cetus to evaluate the security and correctness of the smart contracts associated with the Cetus - Stable Farming project. The assessment included a comprehensive review of the in-scope smart contracts. The audit was performed using a combination of Manual Review and Static Analysis.

The review process emphasized the following areas:

- Architecture review and threat modeling to understand systemic risks and identify design-level flaws.
- Identification of vulnerabilities through both common and edge-case attack vectors.
- Manual verification of contract logic to ensure alignment with intended design and business requirements.
- Dynamic testing to validate runtime behavior and assess execution risks.
- Assessment of code quality and maintainability, including adherence to current best practices and industry standards.

The audit resulted in findings categorized across multiple severity levels, from informational to critical. To enhance the project's security and long-term robustness, we recommend addressing the identified issues and considering the following general improvements:

- Improve code readability and maintainability by adopting a clean architectural pattern and modular design.
- Strengthen testing coverage, including unit and integration tests for key functionalities and edge cases.
- Maintain meaningful inline comments and documentations.
- Implement clear and transparent documentation for privileged roles and sensitive protocol operations.
- Regularly review and simulate contract behavior against newly emerging attack vectors.

REVIEW NOTES | CETUS - STABLE FARMING

Overview

Cetus - Stable Farming is a Sui Move protocol that enables staking of Cetus CLMM position NFTs into on-chain farming pools to earn emissions in one or more reward tokens. Each position's share is computed by projecting its liquidity within a governed effective tick range and valuing token A as token B using a reference sqrt price, a model intended for stable or tightly pegged pairs. Rewards accrue per pool through a reward manager that distributes per-second emissions by allocation points and settles using a standard accPerShare/rewardDebt accounting flow on deposit, harvest, liquidity changes, and withdrawal. Operators govern pool creation, effective ranges, the reference price, and rewarders. Users deposit CLMM positions, harvest rewards, and withdraw.

External Dependencies

The project is developed using the Move language and running on the top of the Sui blockchain. The vulnerability and the updates of the language/Sui framework may affect the project as a whole. As the Sui network is rapidly evolving, to avoid any potential compatibility issues and take advantage of new features and improvements, the client should upgrade the Sui framework to the most recent version. Additionally, staying informed about any upcoming updates or changes to the language or framework can help ensure the project remains secure and compatible.

This protocol also depends on the `cetus_clmm` module for several key implementations, as specified below:

```
[dependencies]
CetusClmm = { local = "../cetus_clmm" }
```

The `cetus_clmm` dependency is **outside the scope of this audit** and is treated as a **trusted external component**. All modules and contracts within this dependency are assumed to be valid, non-vulnerable, and correctly implemented. For the purposes of this audit, they are considered to interact properly with both the current project and other integrated modules.

Privileged Roles

To set up the project correctly and ensure that the project functions properly, owners of the following objects are able to use privileged functions, more details in **Centralization Related Risks and Upgradability**.

The advantage of the privileged role in the codebase is that the client reserves the ability to adjust the protocol according to the runtime required to serve the community best. It is also worthy of note the potential drawbacks of these functions, which should be clearly stated through the client's action/plan. Additionally, if the key pairs of privileged accounts are compromised, the project could have devastating consequences.

To improve the trustworthiness of the project, dynamic runtime updates in the project should be notified to the community. Furthermore, any plan to invoke the aforementioned functions should also be considered to move to the execution queue of the `Timelock` contract.

Upgradeability

Developers working with the Sui blockchain have the ability to upgrade packages based on their software iteration requirements. However, this also means that the `UpgradeCap` and publisher's key store should be handled with caution to prevent any unexpected loss. Additionally, it is important to inform the community about any upgrade plans to address concerns related to centralization and ensure transparency.

Reference:

- Sui Package Upgrades
- Custom Policies

FINDINGS | CETUS - STABLE FARMING



16	0	1	0	2	4	9
Total Findings	Critical	Centralization	High	Medium	Low	Informational

This report has been prepared for Cetus to identify potential vulnerabilities and security issues within the reviewed codebase. During the course of the audit, a total of 16 issues were identified. Leveraging a combination of Manual Review & Static Analysis the following findings were uncovered:

ID	Title	Category	Severity	Status
CSF-08	**Centralization Related Risks And Upgradability**	**Centralization**	**Centralization**	**Acknowledged**
CSF-09	Stale Pool Last_reward_time When Emission=0 Causes Retroactive Rewards	Volatile Code	Medium	Resolved
CSF-11	`linked_table.next()` On A Removed Key Can Aborts	Volatile Code	Medium	Resolved
CSF-12	Inconsistency In Updates Made By `update_effective_tick_range`	Design Issue	Low	Acknowledged
CSF-13	The Risk Of Insufficient Reward Funds	Denial of Service, Design Issue	Low	Acknowledged
CSF-14	Potential Stalled Traversal	Volatile Code	Low	Resolved
CSF-19	Fixed-Coin Liquidity Addition Accepts Unbounded Liquidity And Price Outcomes	Financial Manipulation	Low	Acknowledged
CSF-03	Discussion On "Set_package_version()"	Access Control	Informational	Resolved
CSF-04	Potential Loss Of The Payout	Volatile Code	Informational	Acknowledged
CSF-05	Discussion On Pausing Of `clmm_pool`	Design Issue	Informational	Resolved

ID	Title	Category	Severity	Status
CSF-06	Potential Inconsistent Price Source During Post-Withdraw Validation And Share Accounting	Volatile Code	Informational	Acknowledged
CSF-07	Concerns About Share Calculation Rounds Up	Volatile Code	Informational	Resolved
CSF-10	Several Issues With Migrating Invalid Pool Positions	Inconsistency, Logical Issue	Informational	Resolved
CSF-15	Wrong Error Code	Coding Issue	Informational	Resolved
CSF-16	Wrong `UpdateEffectiveTickRangeEvent` End	Coding Issue	Informational	Resolved
CSF-17	Potential CLMM Pool Mismatch In Migration	Volatile Code	Informational	Resolved

CSF-08 | Centralization Related Risks And Upgradability

Category	Severity	Location	Status
Centralization	**Centralization**		**Acknowledged**

Description

The role **AdminCap** has authority over the functions:

- config::add_operator
- config::set_roles
- config::add_role
- config::remove_role
- config::remove_member
- config::set_package_version
- rewarder::emergent_withdraw
- router::set_roles
- router::add_role
- router::remove_role
- router::add_operator
- router::emergent_withdraw
- config::emergency_unpause

The role **ACL_POOL_MANAGER** has authority over the functions:

- pool::create_pool
- pool::init_invalid_pool_positions
- pool::migrate_invalid_pool_positions
- pool::update_effective_tick_range
- pool::add_rewarder
- pool::update_pool_allocate_point
- router::create_pool
- router::update_effective_tick_range
- router::add_rewarder
- router::update_pool_allocate_point

The role **ACL_REWARDER_MANAGER** has authority over the functions:

- rewarder::create_rewarder
- rewarder::update_rewarder

- router::create_rewarder
- router::update_rewarder

The role **ACL_EMERGENCY_PAUSE** has authority over the functions:

- config::emergency_pause

Note:

- emission_per_second has no upper bound in create_rewarder() / update_rewarder(). It should be set carefully to avoid extreme values; an excessively large rate may cause reward settlement to overflow and block user operations.
- All pools under the same rewarder share a single reward coin balance in the vault. The ACL_REWARDER_MANAGER and ACL_POOL_MANAGER authorities (emission rate and allocation points) should be carefully managed, as they determine how this shared balance is distributed across pools.

The display information can be updated via sui::display module. For example, the deployer can update the properties such as "image_url" and "project_url" in the future.

In addition, developers working with the Sui blockchain can upgrade packages based on their software iteration requirements. However, this also means that the `UpgradeCap` and deployer's key store should be handled carefully to prevent any unexpected losses. It is important to inform the community about any upgrade plans to address concerns related to centralization and ensure transparency.

More information can be found:

- Sui Package Upgrades
- Third-Party Package upgrades

Recommendation

The risk describes the current project design and potentially makes iterations to improve in the security operation and level of decentralization, which in most cases cannot be resolved entirely at the present stage. We advise the client to carefully manage the privileged account's private key to avoid any potential risks of being hacked. In general, we strongly recommend centralized privileges or roles in the protocol be improved via a decentralized mechanism or smart-contract-based accounts with enhanced security practices, e.g., multisignature wallets.

Indicatively, here are some feasible suggestions that would also mitigate the potential risk at a different level in terms of short-term, long-term and permanent:

Short Term:

Timelock and Multi sign (⅔, ⅗) combination *mitigate* by delaying the sensitive operation and avoiding a single point of key management failure.

- Time-lock with reasonable latency, e.g., 48 hours, for awareness on privileged operations;
  AND
- Assignment of privileged roles to multi-signature wallets to prevent a single point of failure due to the private key compromised;

AND

- A medium/blog link for sharing the timelock contract and multi-signers addresses information with the public audience.

Long Term:

Timelock and DAO, the combination, *mitigate* by applying decentralization and transparency.

- Time-lock with reasonable latency, e.g., 48 hours, for awareness on privileged operations;
  AND
- Introduction of a DAO/governance/voting module to increase transparency and user involvement.
  AND
- A medium/blog link for sharing the timelock contract, multi-signers addresses, and DAO information with the public audience.

Permanent:

Renouncing the ownership or removing the function can be considered *fully resolved*.

- Renounce the ownership and never claim back the privileged roles.
  OR
- Remove the risky functionality.

Alleviation

[Cetus, 11/21/2025]:

- **AdminCap** and **UpgradeCap** are already managed by **multi-signature wallets**, ensuring that any privileged or sensitive operations require multiple independent approvals.
- All **ACL roles** (such as ACL_POOL_MANAGER, ACL_REWARDER_MANAGER, ACL_EMERGENCY_PAUSE, etc.) are strictly controlled using **hardware wallets**, reducing the risk of private key compromise.
- We fully acknowledge the importance of security operations and decentralization, and we will continue to optimize governance and key-management policies as the protocol evolves.

We appreciate the suggestions on timelocks, multi-sig workflows, and long-term decentralization mechanisms, and will incorporate these into future roadmap considerations.

[CertiK, 11/21/2025]: It is suggested to implement the aforementioned methods to avoid centralized failure. The finding will be updated when further information is provided.

CSF-09 Stale Pool Last_reward_time When Emission=0 Causes Retroactive Rewards

Category	Severity	Location	Status
Volatile Code	Medium	sources/rewarder.move (0e288bfd710883e9dc48fff439cb6c62b95e57ce): 457~459	Resolved

Description

Rewarder tracks `emission_per_second`, `last_reward_time`, `total_allocate_point`, and aggregate counters. Each pool has `PoolRewarderInfo` with its own `last_reward_time`, `acc_per_share`, and `reward_released` for the pool-wise reward calculation.

In `set_pool`:

- settle pools: `accumulate_pools_reward`
- update rewarder: `accumulate_rewarder_released`

However, when `rewarder.emission_per_second == 0` the `accumulate_pools_reward` will return earlier which will trigger the `last_reward_time` in `accumulate_pool_reward`.

```
        if (rewarder.emission_per_second == 0) {
            return
        };
```

As a result, when `emission_per_second` becomes larger than 0, `accumulate_pool_reward` computes rewards over the entire paused interval using the current (non-zero) emission, crediting pools for time when emission was zero.

Recommendation

Recommend reconsidering the logics when `emission_per_second == 0`.

Alleviation

[Cetus, 11/21/2025]: The team heeded the advice and resolved the issue by updating the logic when `rewarder.emission_per_second == 0` in commit fa08737a605c78c2aa77fe31fb864878c732e469

CSF-11 | `linked_table.next()` On A Removed Key Can Aborts

Category	Severity	Location	Status
Volatile Code	Medium	sources/pool.move (update): 353	Resolved

Description

The code removes the current key and then calls `linked_table::next` on that removed key. Per Sui's `linked_table` implementation, `next` aborts if the key does not exist (it only returns None when the key exists but has no successor, i.e., tail).

As a result, the function will be abort, since the `wrapped_pos_id` has already been removed.

```
        let position_info = pool.positions.remove(wrapped_pos_id);
        ...
        next_id = *linked_table::next(&pool.positions, wrapped_pos_id);
```

Scenario

The following minimum test is trying to describe the linked_table remove-then-next scenario:

```
#[test]
#[expected_failure]
fun test_linked_table_next_after_remove_panics() {
    // Minimal reproducer: calling next on a removed key is invalid.
    let mut scenario_ = test_scenario::begin(ADMIN);
    let scenario = &mut scenario_;
    {
        let ctx = test_scenario::ctx(scenario);
        let mut table = linked_table::new(ctx);
        linked_table::push_back(&mut table, 1, 10);
        linked_table::push_back(&mut table, 2, 20);
        // Take the first key
        let first = *linked_table::front(&table);
        let k = *option::borrow(&first);
        // Safe while key exists
        let _ = linked_table::next(&table, k);
        // Remove current key
        let _ = linked_table::remove(&mut table, k);
        // This is the problematic pattern: next on a removed key -> abort expected
        let _ = linked_table::next(&table, k);
        return_shared(table);
    };
    test_scenario::end(scenario_);
}
```

Recommendation

Recommend reconsidering the pointer logics for the batch mitigation and adding corresponding tests.

Alleviation

[Cetus, 11/25/2025]: The team heeded the advice and resolved the issue by using `next()` method before any removal operation in commit fa08737a605c78c2aa77fe31fb864878c732e469.

CSF-12 Inconsistency In Updates Made By `update_effective_tick_range`

Category	Severity	Location	Status
Design Issue	Low	sources/pool.move (0e288bfd710883e9dc48fff439cb6c62b95e57ce): 326~327	Acknowledged

Description

When updating `effective_tick_lower`, `effective_tick_upper`, and `sqrt_price` via `update_effective_tick_range`, due to gas limitations, if a pool contains too many `Pool.positions`, the update must be divided into multiple `update_effective_tick_range` transactions.

However, if these batched transactions are not executed at the **same time**, the `Pool.positions` updated later will have inconsistent share values during the interval between the first and the later transactions. This happens because the pool's `effective_tick_lower`, `effective_tick_upper`, and `sqrt_price` have already been updated to new values, while those positions updating later are still using outdated shares.

In this situation, such positions may receive more or fewer rewards than intended.

Recommendation

We recommend reviewing the design of `update_effective_tick_range`.

Alleviation

[Cetus, 11/25/2025]: Due to sui limits, it is indeed not feasible to update all position shares within a single update_effective_tick_range transaction. This means that during the brief batching window, some positions will temporarily calculate shares using the old parameters, while others use the updated ones. As a result, rewards generated only during this short interval may be distributed with slight inconsistency.

However, this impact is limited and acceptable for several reasons:

The inconsistency window is extremely short, only spanning the time required to complete all batched update transactions. Only rewards generated within this small window are affected, which is negligible compared to the entire reward distribution over the farming period. There is no practical alternative to update all position shares in a single transaction given gas constraints, so batching is necessary for system scalability. Given these constraints and the minimal economic impact, the design is considered acceptable.

CSF-13 | The Risk Of Insufficient Reward Funds

Category	Severity	Location	Status
Denial of Service, Design Issue	Low	sources/rewarder.move (0e288bfd710883e9dc48fff439cb6c62b95e57ce): 108, 130	Acknowledged

Description

Currently, the contract lacks an economic mechanism to obtain reward funds autonomously; reward funds are injected externally via `deposit_rewarder()` and can also be withdrawn via `emergent_withdraw` by addresses holding `AdminCap` privileges. If the contract's reward funds become insufficient, users may experience a denial-of-service (DoS) when interacting with the contract.

Note that withdraw() / close_position_v2() require all farming rewards to be harvested first. If the vault is insufficient (or the reward coin was never funded), users can still remove most of their liquidity via remove_liquidity(), but the wrapped position and its residual liquidity cannot be withdrawn until the vault is replenished.

Recommendation

We recommend the project implement a self-sustaining economic mechanism to supply the required reward funds. If the original design is retained, ensure there are sufficient reward funds to prevent DoS.

Alleviation

[Cetus, 11/21/2025]: We acknowledge that the reward pool is externally funded and can be withdrawn via `emergent_withdraw` by the AdminCap holder. From an operational perspective, this does not introduce practical DoS risk for users: the team ensures that reward funds are continuously sufficient to support all active farming programs, and withdrawals are strictly managed under internal operational procedures.

At this stage, the reward model is intentionally designed to be externally funded rather than self-sustaining. We will continue to monitor reward utilization and maintain adequate reward balances to prevent any interruption in user operations.

CSF-14 | Potential Stalled Traversal

Category	Severity	Location	Status
Volatile Code	Low	sources/pool.move (update): 335~349	Resolved

Description

When a position is "kept" with `amount_a > 0 or amount_b > 0`, the loop increments count but never advances `next_id`. This causes the loop to repeatedly re-check the same node until limit is reached, preventing traversal toward the tail and potentially starving later entries.

As a result, the batch migration does not progress and the last positions may never be visited.

Recommendation

Recommend reconsider the logic with "keep" branch and update the `next_id` as well.

Alleviation

[Cetus, 11/25/2025]: The team heeded the advice and resolved the issue by revising the function logic in commit fa08737a605c78c2aa77fe31fb864878c732e469.

CSF-19 Fixed-Coin Liquidity Addition Accepts Unbounded Liquidity And Price Outcomes

Category	Severity	Location	Status
Financial Manipulation	Low	sources/router.move (update-101526): 277~278	Acknowledged

Description

The `stable_farming::router::add_liquidity_fix_coin` function allows a user to increase the liquidity of an existing wrapped CLMM position by fixing the input amount of either CoinA or CoinB while treating the other token amount as a maximum spending limit. Any unused tokens are returned to the caller. However, the function does not allow the caller to specify a minimum acceptable liquidity increase, an execution-price limit, or a transaction deadline.

In `stable_farming/sources/router.move:277-290`, the entry function accepts only the two token amounts and the fixed-token selector:

```
public entry fun add_liquidity_fix_coin<CoinA, CoinB>(
    ...
    amount_a: u64,
    amount_b: u64,
    fix_amount_a: bool,
    clk: &Clock,
    ctx: &mut TxContext,
)
```

The selected fixed-token amount is forwarded to the underlying CLMM, which calculates the resulting liquidity using the pool price at the time the transaction executes. When CoinA is fixed and the position is currently in range, `current_sqrt_price` is used directly to derive the liquidity and the required amount of CoinB.

```
// cetus_clmm/sources/math/clmm_math.move:426-436
if (is_fixed_a) {
    assert!(current_sqrt_price < upper_price, EINVALID_FIXED_TOKEN_TYPE);
    if (current_sqrt_price <= lower_price) {
        _liquidity = get_liquidity_from_a(lower_price, upper_price, amount, false);
    } else {
        // Both A and B
        _liquidity = get_liquidity_from_a(
            current_sqrt_price,
            upper_price,
            amount,
            false,
        );
```

After the CLMM calculates the result, the stable-farming contract only verifies that the required amounts of CoinA and CoinB do not exceed the caller-provided limits. It does not verify that the actual liquidity increase meets a minimum value acceptable to the caller.

```
// stable_farming/sources/pool.move:1559-1575
let amount = if (fix_amount_a) { amount_a } else { amount_b };
let receipt = clmm_pool::add_liquidity_fix_coin<CoinA, CoinB>(
    clmm_config,
    clmm_pool,
    clmm_position_nft,
    amount,
    fix_amount_a,
    clock,
);
let (amount_need_a, amount_need_b) =
    clmm_pool::add_liquidity_pay_amount(&receipt);
assert!(amount_need_a <= amount_a, EAmountInAboveMaxLimit);
assert!(amount_need_b <= amount_b, EAmountInAboveMaxLimit);
```

An attacker can observe a pending fixed-coin liquidity-addition transaction and execute a swap immediately before it. The swap moves the CLMM price in the direction that reduces the liquidity derived from the victim's fixed-token input. In the affected direction, the required amount of the non-fixed token may also decrease, allowing both maximum-input checks to pass. The victim's transaction consequently succeeds despite receiving substantially less liquidity than expected. The attacker can then execute an inverse swap after the victim's transaction to restore the price.

For example, consider a position with normalized sqrt-price bounds of approximately `0.7071` and `1.4142`. At a current sqrt price of `1`, fixing `100` CoinA produces approximately `341.4` units of liquidity and requires approximately `100` CoinB. If an attacker first moves the sqrt price to `0.8`, the same fixed input produces only approximately `184.2` units of liquidity and requires approximately `17.1` CoinB. A transaction allowing up to `100` CoinB still passes because `17.1 <= 100`, even though the victim receives approximately 46% less liquidity.

Consequently, the user's full fixed-token input may be committed at the manipulated price in exchange for substantially less liquidity than expected. As a result, after the price is restored, the victim is left with a lower-value liquidity position and suffers a direct financial loss, while the attacker may capture part of the lost value as sandwich profit.

Recommendation

Add a caller-defined `min_delta_liquidity` parameter to the fixed-coin liquidity-addition path and propagate it through the router and pool interfaces. Before completing repayment, calculate the actual increase in position liquidity and abort the transaction when it is below the caller's specified minimum.

The interface should also support a caller-defined sqrt-price boundary and transaction deadline to prevent execution after the price has moved outside the user's acceptable range or after the transaction has become stale.

Alleviation

[Cetus, 10/20/2026]: We appreciate you bringing this point to our attention. We confirm that the fixed-coin liquidity addition flow may be affected by adverse price movement, as the current parameters only enforce maximum input amounts.

We plan to add price-slippage protection in a future update. The transaction builder will include an additional on-chain Move call, executed in the same PTB immediately before the liquidity-addition call, to verify that the pool price remains within the user-defined slippage range. The transaction will abort if the price is outside the permitted range.

CSF-03 | Discussion On "Set_package_version()"

Category	Severity	Location	Status
Access Control	Informational	sources/config.move (0e288bfd710883e9dc48fff439cb6c62b95e57ce): 169~176	Resolved

Description

`set_package_version()` allows the version to be set to any value, meaning that the function's version can be set not only to `EMERGENCY_RESTORE_NEED_VERSION`, `EMERGENCY_PAUSE_VERSION`, and `VERSION`, but also to any previous version.

`config.move`

```
169     public fun set_package_version(_: &AdminCap, config: &mut GlobalConfig,
version: u64) {
170         let old_version = config.package_version;
171         config.package_version = version;
172         emit(SetPackageVersion {
173             new_version: version,
174             old_version
175         })
176     }
```

Since there is no version control, the code for past versions of `set_package_version()` will remain permanently available.

This would allow the version to be set back to any previous version. Additionally, the functionality of `emergency_pause()` can also be achieved through `set_package_version()`.

We would like to confirm with the official team if this is the intended design.

Recommendation

We would like to confirm with the official team if this is the intended design.

Alleviation

[Cetus, 11/21/2025]: The team heeded the advice and resolved the issue by adding version control in commit fa08737a605c78c2aa77fe31fb864878c732e469.

[CertiK, 11/21/2025]:

As a follow-up regarding the emergency pause: since the current release does not provide a way to set `config.version` to `EMERGENCY_RESTORE_NEED_VERSION` or to unpause the functions, the system will remain paused until a new package is published that includes the logic associated with `EMERGENCY_RESTORE_NEED_VERSION`.

Additionally, we want to emphasize that the new package **should not** set its `VERSION` to `EMERGENCY_RESTORE_NEED_VERSION` directly. Doing so would cause `checked_package_version` to behave incorrectly: because `VERSION >= config.package_version` will always evaluate to true (given that `EMERGENCY_RESTORE_NEED_VERSION` is larger than all normal versions), the package would remain permanently valid. This means that even after upgrading `config.package_version` back to a normal value (e.g., 12), the emergency package would still be accepted unintentionally.

We would like to double-check with the team regarding the scenario above, and if the process is intended.

[Cetus, 11/26/2025]:

We've added an **unpause method** so the system can properly resume after an emergency.

And yes, we confirm that packages should **not** set their VERSION to `EMERGENCY_RESTORE_NEED_VERSION` ; it's only meant to trigger the emergency path.

https://github.com/CetusProtocol/cetus-clmm-sui/pull/45/commits/14fadb735e3ec744721fe65e05f72554045edeca

---

[CertiK, 11/27/2025]: The emergency_unpause function may allow the project to revert to an earlier package version rather than only moving forward to a version higher than the latest normal release.

For example, if both package versions A and B include emergency_unpause, the admin could unpause the config to either A or B.

This introduces operational risk and therefore requires careful attention from the admin during the process.

[Cetus, 12/01/2025]: We will take this into account. When performing an emergency unpause, our operational procedure will ensure that the config is restored only to the latest correct and intended version, avoiding the risk of reverting to an outdated package.

CSF-04 | Potential Loss Of The Payout

Category	Severity	Location	Status
Volatile Code	Informational	sources/pool.move (0e288bfd710883e9dc48fff439cb6c62b95e57ce): 705~706	Acknowledged

Description

The internal accounting stores pending rewards in base units while harvest divides by `REWARD_PRECISION` before paying and then zeroes the pending counter.

```
        let reward_amount = ((pending_reward / REWARD_PRECISION) as u64);
        reward_info.reward = 0;
```

However, there is no validation if the reward_amount is 0 or not, if the user invokes the function not exceed the scale factor, the reward will be erased.

Recommendation

We would like to check with the team if the scenario has been considered.

Alleviation

[Cetus, 11/25/2025]: Yes, this scenario has been considered. The pending_reward value is already stored in base units scaled by 10^13 (i.e., REWARD_PRECISION). When converting it back to the actual reward amount during harvest, the truncation of residual precision (resulting in reward_amount = 0 for very small values) is acceptable by design.

These tiny residual amounts are far below the intended payout granularity, and losing them does not materially impact user rewards nor the overall reward distribution logic.

CSF-05 | Discussion On Pausing Of `clmm_pool`

Category	Severity	Location	Status
Design Issue	Informational	sources/pool.move (0e288bfd710883e9dc48fff439cb6c62b95e57ce): 85~95	Resolved

Description

The `clmm_pool` can be paused.

```
143 public struct Pool<phantom CoinTypeA, phantom CoinTypeB> has key, store {
144     id: UID,
145     coin_a: Balance<CoinTypeA>,
146     coin_b: Balance<CoinTypeB>,
147     tick_spacing: u32,
148     fee_rate: u64,
149     liquidity: u128,
150     current_sqrt_price: u128,
151     current_tick_index: I32,
152     fee_growth_global_a: u128,
153     fee_growth_global_b: u128,
154     fee_protocol_coin_a: u64,
155     fee_protocol_coin_b: u64,
156     tick_manager: TickManager,
157     rewarder_manager: RewarderManager,
158     position_manager: PositionManager,
159     is_pause: bool,
160     index: u64,
161     url: String,
162 }
```

However, the stable farming pool can not be paused individually, nor will its corresponding operations be disabled when the `clmm_pool` is paused.

```
85     public struct Pool has key, store {
86         id: UID,
87         clmm_pool_id: ID,
88         effective_tick_lower: I32,
89         effective_tick_upper: I32,
90         // The sqrt price(Q64X64) of CoinA
91         sqrt_price: u128,
92         total_share: u128,
93         rewarders: vector<TypeName>,
94         positions: LinkedTable<ID, WrappedPositionInfo>
95     }
```

In addition, after the `clmm_pool` is paused, the rewarder continues to distribute rewards over time based on `emission_per_second`.

Recommendation

We would like to confirm with the official team whether this is the intended design.

Alleviation

[Cetus, 11/25/2025]: Yes, this behavior is intended. Even if the CLMM pool is paused, the rewarder can effectively stop distributing rewards by setting emission_per_second to 0, which halts further reward accumulation. This provides the protocol with full control to stop emissions when needed without modifying other pool states.

CSF-06 Potential Inconsistent Price Source During Post-Withdraw Validation And Share Accounting

Category	Severity	Location	Status
Volatile Code	Informational	sources/pool.move (update): 1063~1075	Acknowledged

Description

After removing liquidity, the position validity check uses the CLMM's live price to ensure the position **still** has liquidity.

```
    assert_position_liquidity_valid(
        clmm_pool.current_sqrt_price(),
        position_tick_lower,
        position_tick_upper,
        new_liquidity,
    );
```

while share recalculation uses the farming pool's stored pool.sqrt_price.

```
    // Recalculate the position share
    let new_pos_share = calculate_position_share(
        wrapped_pos_info.tick_lower,
        wrapped_pos_info.tick_upper,
        wrapped_pos_info.liquidity,
        pool.sqrt_price,
        pool.effective_tick_lower,
        pool.effective_tick_upper,
    );
```

If these diverge, the validity check can abort even when the position would still hold a non-zero share under `pool.sqrt_price`.

Recommendation

We would like check with the team if the scenario has been considered and the timing for updating `pool.sqrt_price`.

Alleviation

[Cetus, 11/25/2025]: Yes, this behavior is intentional.

The position-validity check and share calculation serve different purposes and therefore use different prices:

Position validity is determined using the current CLMM pool price, ensuring that the position still contains real, withdrawable token A/B amounts based on live market conditions. Share calculation uses the farming pool's stored sqrtprice and effective

tick range, which are predefined parameters and are applied uniformly to all positions for reward distribution.

Because these two mechanisms address different objectives, their calculations do not need to align. Divergence between the current CLMM price and the farming pool's sqrtprice does not affect fairness or correctness.

CSF-07 | Concerns About Share Calculation Rounds Up

Category	Severity	Location	Status
Volatile Code	Informational	sources/pool.move (update): 1293~1300	Resolved

Description

`calculate_position_share` computes token amounts from liquidity by rounding up. This bias tends to increase and can slightly inflate the computed share, especially when one position is split into multiple tiny positions.

```
    let (amount_a, amount_b) = get_amount_by_liquidity(
        valid_tick_lower,
        valid_tick_upper,
        tick_math::get_tick_at_sqrt_price(sqrt_price),
        sqrt_price,
        liquidity,
        true,
    );
```

Recommendation

We would like to check with the team about the purpose of the rounding-up operation.

Alleviation

[Cetus, 11/25/2025]: The team heeded the advice and resolved the issue by setting the flag to `false` in commit fa08737a605c78c2aa77fe31fb864878c732e469.

CSF-10 | Several Issues With Migrating Invalid Pool Positions

Category	Severity	Location	Status
Inconsistency, Logical Issue	Informational	sources/pool.move (update): 351, 691~708	Resolved

Description

The `migrate_invalid_pool_positions` function is designed to migrate old invalid positions—those with zero liquidity (i.e., both `amount_a` and `amount_b` are 0)—to `invalid_pool_positions.positions`, and allows users to withdraw these invalid positions via `withdraw()`. However, there are two issues during the migration process:

1. **Rewards not settled during migration:** The function does not settle the pool rewards or position rewards at the current timestamp. Although the position has zero liquidity and therefore zero share at this point—meaning its rewards should not change—`migrate_invalid_pool_positions()` still modifies `pool.total_share`. It is unclear whether this behavior is intended by the development team.

`pool.move`

```
350      let position_info = pool.positions.remove(wrapped_pos_id);
351  @>  pool.total_share = pool.total_share - position_info.share;
352      invalid_pool_positions.positions.push_back(wrapped_pos_id, position_info);
353      next_id = *linked_table::next(&pool.positions, wrapped_pos_id);
354      count = count + 1;
```

If modifying `pool.total_share` is indeed necessary, the process should follow the same logic as elsewhere, i.e., through `rewarder::pool_rewards_settle()` -> `update_position_share()` -> `pool.total_share = new_pool_share` -> `rewarder::set_pool_share`, to properly settle and update both the pool and position rewards.

2. **Rewards not withdrawn:** When withdrawing, the function does not extract or check the rewards of the invalid position. This could lead to the position's rewards being locked in the contract.

`pool.move`

```
691     if (!pool.positions.contains(wrapped_position_id)) {
692         let invalid_pool_positions = dof::borrow_mut<String,
PoolInvalidPositions>(
693             &mut pool.id,
694             string::utf8(INVALID_POOL_POSITIONS_KEY),
695         );
696
697         let wraped_position = invalid_pool_positions.positions.remove(
wrapped_position_id);
698         wraped_position.destory_wrapping_position_info();
699
700         event::emit(WithdrawEvent {
701             pool_id: object::id(pool),
702             wrapped_position_id,
703             clmm_pool_id,
704             clmm_position_id: object::id(&position),
705             share: 0,
706         });
707         return position
708     };
```

Recommendation

For issue (1), we want to confirm the intended design with the development team and adjusting the logic accordingly.

For issue (2), we suggest returning the corresponding rewards to the position owner before withdrawing the `clmm_position` from `pool_position` during `withdraw()`.

Alleviation

[Cetus, 11/25/2025]: The team heeded the advice and resolved the issue by adding logic to handle invalid positions in the `withdraw()` and `harvest()` functions in commit fa08737a605c78c2aa77fe31fb864878c732e469.

CSF-15 | Wrong Error Code

Category	Severity	Location	Status
Coding Issue	Informational	sources/rewarder.move (0e288bfd710883e9dc48fff439cb6c62b95e57ce): 319	Resolved

Description

In `withdraw_reward()`, the `is_pool_in_rewarder()` check uses the wrong error code `EClmmPoolNotExist`.

`rewarder.move`

```
 319         assert!(is_pool_in_rewarder(manager, type_name::with_defining_ids<
RewardCoin>(), clmm_pool_id), EClmmPoolNotExist);
```

It should use the correct error code `EClmmPoolNotInRewarder`, consistent with the meaning used in other parts.

`rewarder.move`

```
 261         assert!(is_pool_in_rewarder(manager, type_name::with_defining_ids<
RewardCoin>(), clmm_pool_id), EClmmPoolNotInRewarder);
```

Recommendation

We recommend using the same or consistent error codes.

Alleviation

[CertiK, 11/21/2025]:

Thanks for the update. The following function shares the wrong error code, as the assertion is intended to avoid a pool already in the rewarder.

```
public(package) fun add_pool<RewardCoin>(
    manager: &mut RewarderManager,
    clmm_pool_id: ID,
    allocate_point: u64,
    clock: &Clock,
) {
    ...
    assert!(
        !is_pool_in_rewarder(manager, type_name::with_defining_ids<RewardCoin>(),
clmm_pool_id),
        EClmmPoolNotInRewarder,
    );
    ...
}
```

[Cetus, 11/27/2025]: The team heeded the advice and resolved the issue by updating the error code in commit 14fadb735e3ec744721fe65e05f72554045edeca.

CSF-16 | Wrong `UpdateEffectiveTickRangeEvent` End

Category	Severity	Location	Status
Coding Issue	Informational	sources/pool.move (0e288bfd710883e9dc48fff439cb6c62b95e57ce): 429	Resolved

Description

In the `UpdateEffectiveTickRangeEvent` emitted by `update_effective_tick_range_internal`, the `end` field currently uses the last element of `pool.positions`, rather than the actual last processed element. This behavior is inconsistent with the intended design.

`pool.move`

```
429         let end = *linked_table::back(&pool.positions);
430         event::emit(UpdateEffectiveTickRangeEvent {
431             pool_id: object::id(pool),
432             clmm_pool_id: object::id(clmm_pool),
433             effective_tick_lower,
434             effective_tick_upper,
435             sqrt_price,
436             start,
437             end,
438             limit
439         });
```

Recommendation

We recommend retaining the ID of the last processed element and using that ID as the `end`.

Alleviation

[Cetus, 11/25/2025]: The team heeded the advice and resolved the issue by using the correct ID as the `end` in commit fa08737a605c78c2aa77fe31fb864878c732e469.

CSF-17 | Potential CLMM Pool Mismatch In Migration

Category	Severity	Location	Status
Volatile Code	Informational	sources/pool.move (update): 314~315	Resolved

Description

The function `migrate_invalid_pool_positions` derives `current_sqrt_price` with input `clmm_pool`, so if the passed `clmm_pool` does not belong to the farming `pool`, positions can be incorrectly treated as invalid/valid due to using the wrong price.

Recommendation

Recommend adding validation between the given `clmm_pool` and `pool`.

Alleviation

[Cetus, 11/21/2025]: The team heeded the advice and resolved the issue by adding pool-id checks in commit fa08737a605c78c2aa77fe31fb864878c732e469.

OPTIMIZATIONS | CETUS - STABLE FARMING

ID	Title	Category	Severity	Status
CSF-01	Lack `clmm_pool` Id Check	Coding Style	Optimization	Resolved
CSF-02	High Gas Consumption Issues In Pool And Rewarder	Gas Optimization	Optimization	Acknowledged

CSF-01 | Lack `clmm_pool` Id Check

Category	Severity	Location	Status
Coding Style	Optimization	sources/pool.move (0e288bfd710883e9dc48fff439cb6c62b95e57ce): 534	Resolved

Description

The following functions are missing explicit pool-id assertions:

- `deposit_v2`
- `close_position_v2`

Recommendation

Recommend adding explicit check for clmm_pool and pool(farming pool).

Alleviation

[Cetus, 11/21/2025]: The team heeded the advice and resolved the issue by adding pool-id checks in commit fa08737a605c78c2aa77fe31fb864878c732e469.

CSF-02 | High Gas Consumption Issues In Pool And Rewarder

Category	Severity	Location	Status
Gas Optimization	Optimization	sources/pool.move (update): 1393~1418	Acknowledged

Description

In both the pool and rewarder, there are two aspects that may lead to high gas consumption:

1. The `rewards` field in `WrappedPositionInfo` uses a `VecMap` structure. Because of this, the `vec_map::get_mut` and `vec_map::contains` calls in `update_position_share` each involve a loop. Combined with the existing `while` loop in the function, this results in a nested loop. When the lengths of `pool_acc_per_shares` or `wrapped_pos_info.rewards` are large, gas costs increase significantly.

`pool.move`

```
 1393  @>  while (index < vec_map::length(&pool_acc_per_shares)) {
 1394          let (reward_coin, pool_acc_per_share) = vec_map::get_entry_by_idx(
 1395              &pool_acc_per_shares,
 1396              index,
 1397          );
 1398          let reward_debt = (full_math_u128::full_mul(new_share, *
pool_acc_per_share) as u128);
 1399          let reward_accumulated = (
 1400              full_math_u128::full_mul(wrapped_pos_info.share, *
pool_acc_per_share) as u128,
 1401          );
 1402  @>      if (!vec_map::contains(&wrapped_pos_info.rewards, reward_coin)) {
 1403              vec_map::insert(
 1404                  &mut wrapped_pos_info.rewards,
 1405                  *reward_coin,
 1406                  PositionRewardInfo {
 1407                      reward: reward_accumulated,
 1408                      reward_debt,
 1409                      reward_harvested: 0,
 1410                  },
 1411              );
 1412          } else {
 1413  @>          let reward_info = vec_map::get_mut(&mut wrapped_pos_info.rewards,
 reward_coin);
 1414              reward_info.reward = reward_info.reward + reward_accumulated -
 reward_info.reward_debt;
 1415              reward_info.reward_debt = reward_debt;
 1416          };
 1417          index = index + 1
 1418      }
```

2. In `pool_rewards_settle()` and `accumulate_pools_reward()`, the function `accumulate_pool_reward()` is called to update rewards—`pool_rewards_settle()` for all rewarders associated with a given pool, and `accumulate_pools_reward()` for all pools associated with a given rewarder. However, the contract lack a mechanism to remove a pool from a rewarder or remove a rewarder. Instead, reward distribution can only be paused by setting a pool's `allocate_point` to `0` or a rewarder's `emission_per_second` to `0`. As a result, even when a rewarder is disabled or a pool can no longer claim rewards from that rewarder, the corresponding pool or rewarder entries remain. These entries continue to be iterated over in the aforementioned functions, may causing high gas consumption.

Recommendation

We would like to confirm whether this is the intended design. If not, we recommend that the development team assess the impact and modify the code to reduce gas consumption. If it is intentional, it is important to carefully control the number of rewarders（reward coins） and the number of pools associated with each rewarder.

Alleviation

[Cetus, 11/25/2025]: The current design does result in higher gas usage when a pool or rewarder accumulates a large number of reward entries, mainly due to the VecMap iteration pattern and the fact that disabled pools/rewarders remain in the iteration set.

However, in our production environment this is operationally acceptable, because:

The number of pools and reward tokens is intentionally kept very small.

We do not expect a large reward-token set or a large pool set, so the nested iterations remain bounded and inexpensive in practice.

Removing pools or rewarders now would require substantial structural changes.

As the protocol has already launched, introducing removal mechanisms or redesigning storage structures would involve significant refactoring and migration considerations.

Given the operational scale and constraints, the current implementation is sufficient.

Gas usage remains manageable under the expected configuration, and we prefer stability over large architectural changes at this stage.

We may revisit this optimization if future scaling needs change, but for now, the existing behavior is intentional and acceptable.

APPENDIX | CETUS - STABLE FARMING

Finding Categories

Categories	Description
Centralization	Centralization findings detail the design choices of designating privileged roles or other centralized controls over the code.
Volatile Code	Volatile Code findings refer to segments of code that behave unexpectedly on certain edge cases and may result in vulnerabilities.
Design Issue	Design Issue findings indicate general issues at the design level beyond program logic that are not covered by other finding categories.
Denial of Service	Denial of Service findings indicate that an attacker may prevent the program from operating correctly or responding to legitimate requests.
Financial Manipulation	Financial Manipulation findings indicate issues in design that may lead to financial losses.
Access Control	Access Control findings are about security vulnerabilities that make protected assets unsafe.
Inconsistency	Inconsistency findings refer to different parts of code that are not consistent or code that does not behave according to its specification.
Logical Issue	Logical Issue findings indicate general implementation issues related to the program logic.
Coding Issue	Coding Issue findings are about general code quality including, but not limited to, coding mistakes, compile errors, and performance issues.
Coding Style	Coding Style findings may not affect code behavior, but indicate areas where coding practices can be improved to make the code more understandable and maintainable.
Gas Optimization	"Gas" is used here as generic term in DLT world, that can differ from chain to chain. Finding indicates that computational, storage resources can be saved, for benefit of users and efficiency of chain. Also in some cases, being not resourceful may lead to DoS attacks.

DISCLAIMER | CERTIK

This report is subject to the terms and conditions (including without limitation, description of services, confidentiality, disclaimer and limitation of liability) set forth in the Services Agreement, or the scope of services, and terms and conditions provided to you ("Customer" or the "Company") in connection with the Agreement. This report provided in connection with the Services set forth in the Agreement shall be used by the Company only to the extent permitted under the terms and conditions set forth in the Agreement. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes, nor may copies be delivered to any other person other than the Company, without CertiK's prior written consent in each instance.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts CertiK to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. CertiK's position is that each company and individual are responsible for their own due diligence and continuous security. CertiK's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by CertiK is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

ALL SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF ARE PROVIDED "AS IS" AND "AS AVAILABLE" AND WITH ALL FAULTS AND DEFECTS WITHOUT WARRANTY OF ANY KIND. TO THE MAXIMUM EXTENT PERMITTED UNDER APPLICABLE LAW, CERTIK HEREBY DISCLAIMS ALL WARRANTIES, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHERWISE WITH RESPECT TO THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS. WITHOUT LIMITING THE FOREGOING, CERTIK SPECIFICALLY DISCLAIMS ALL IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, TITLE AND NON-INFRINGEMENT, AND ALL WARRANTIES ARISING FROM COURSE OF DEALING, USAGE, OR TRADE PRACTICE. WITHOUT LIMITING THE FOREGOING, CERTIK MAKES NO WARRANTY OF ANY KIND THAT THE SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF, WILL MEET CUSTOMER'S OR ANY OTHER PERSON'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULT, BE COMPATIBLE OR WORK WITH ANY SOFTWARE, SYSTEM, OR OTHER SERVICES, OR BE SECURE, ACCURATE, COMPLETE, FREE OF HARMFUL CODE, OR ERROR-FREE. WITHOUT LIMITATION TO THE FOREGOING, CERTIK PROVIDES NO WARRANTY OR

UNDERTAKING, AND MAKES NO REPRESENTATION OF ANY KIND THAT THE SERVICE WILL MEET CUSTOMER'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULTS, BE COMPATIBLE OR WORK WITH ANY OTHER SOFTWARE, APPLICATIONS, SYSTEMS OR SERVICES, OPERATE WITHOUT INTERRUPTION, MEET ANY PERFORMANCE OR RELIABILITY STANDARDS OR BE ERROR FREE OR THAT ANY ERRORS OR DEFECTS CAN OR WILL BE CORRECTED.

WITHOUT LIMITING THE FOREGOING, NEITHER CERTIK NOR ANY OF CERTIK'S AGENTS MAKES ANY REPRESENTATION OR WARRANTY OF ANY KIND, EXPRESS OR IMPLIED AS TO THE ACCURACY, RELIABILITY, OR CURRENCY OF ANY INFORMATION OR CONTENT PROVIDED THROUGH THE SERVICE. CERTIK WILL ASSUME NO LIABILITY OR RESPONSIBILITY FOR (I) ANY ERRORS, MISTAKES, OR INACCURACIES OF CONTENT AND MATERIALS OR FOR ANY LOSS OR DAMAGE OF ANY KIND INCURRED AS A RESULT OF THE USE OF ANY CONTENT, OR (II) ANY PERSONAL INJURY OR PROPERTY DAMAGE, OF ANY NATURE WHATSOEVER, RESULTING FROM CUSTOMER'S ACCESS TO OR USE OF THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS.

ALL THIRD-PARTY MATERIALS ARE PROVIDED "AS IS" AND ANY REPRESENTATION OR WARRANTY OF OR CONCERNING ANY THIRD-PARTY MATERIALS IS STRICTLY BETWEEN CUSTOMER AND THE THIRD-PARTY OWNER OR DISTRIBUTOR OF THE THIRD-PARTY MATERIALS.

THE SERVICES, ASSESSMENT REPORT, AND ANY OTHER MATERIALS HEREUNDER ARE SOLELY PROVIDED TO CUSTOMER AND MAY NOT BE RELIED ON BY ANY OTHER PERSON OR FOR ANY PURPOSE NOT SPECIFICALLY IDENTIFIED IN THIS AGREEMENT, NOR MAY COPIES BE DELIVERED TO, ANY OTHER PERSON WITHOUT CERTIK'S PRIOR WRITTEN CONSENT IN EACH INSTANCE.

NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS.

THE REPRESENTATIONS AND WARRANTIES OF CERTIK CONTAINED IN THIS AGREEMENT ARE SOLELY FOR THE BENEFIT OF CUSTOMER. ACCORDINGLY, NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH REPRESENTATIONS AND WARRANTIES AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH REPRESENTATIONS OR WARRANTIES OR ANY MATTER SUBJECT TO OR RESULTING IN INDEMNIFICATION UNDER THIS AGREEMENT OR OTHERWISE.

FOR AVOIDANCE OF DOUBT, THE SERVICES, INCLUDING ANY ASSOCIATED ASSESSMENT REPORTS OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.

Elevate Your Web3 Journey

CertiK is the largest Web3 security platform combining formal verification with audits and comprehensive security solutions.